UNIT-I

Syllabus:

Finite Automata: Central concepts of strings, languages and automata theory, Deterministic Finite Automata, Non-Deterministic Finite Automata, Finite Automata with Epsilon Transitions, Finite Automata with Output.

UNIT-I

1. Central Concepts of Automata Theory:

1.1 Symbol:

A symbol is the smallest building block (an abstract entity), which can be any alphabet, letter, or any picture.

Example: Frequently used symbols are: a, b, c, ……, z, A, B,……,Z, 0, 1, …..9

1.2 Alphabet:

An alphabet is a *finite set of symbols*. It is denoted by **Σ**(Sigma).

Example: Frequently used alphabets are:

- Σ = {0,1} is an alphabet of binary digits
- Σ = {0,1,2,….9} is an alphabet of decimal digits
- Σ = {a,b,c} is one alphabet
- Σ = {A,B,C,….,Z} is an alphabet of Capital Letters
- Σ = {0,1,a,b} is one alphabet

1.3 String:

A string is a *finite sequence of symbols* chosen from some alphabet. It is generally denoted by **w** and string is also called a *word.*

Examples:

- a , b , aa , ab , ba , …… are the strings from the alphabet Σ={a,b}
- 01101 is a string from the binary alphabet Σ={0,1}

1.3.1 Length of a string:

The length of a string is the number of symbols in the string. It is denoted by |w|.

Example: If w=abaa is a string then length is |w|=|abaa|=4.

- **What is the (How many) number of strings of length 2 that can be generated over an alphabet Σ={a,b}.**
  Number of strings of length 2 that can be generated over the alphabet Σ={a,b} is:
  Ans: The length of the string should be 2 i.e _ _ (2 spaces) so for each blank space I have 2 combinations(Because Σ is with 2 symbols) So 2x2 = 4 i.e $\mathbf{2^2}$

  Strings are: aa , ab, ba, bb total 4.

- **Number of strings of length 3 that can be generated over an alphabet Σ={a,b} is:**
  Ans: The length of the string should be 3 i.e _ _ _ (3 spaces) so for each blank space I have 2 combinations(Because Σ is with 2 symbols) So 2x2x2 = 8 i.e $\mathbf{2^3}$

  Strings are: aaa , aab, aba, abb, baa, bab, bba, bbb total 8.

- **Similarly Number of strings of length n that can be generated over the alphabet Σ={a,b} is:**
  Ans: The length of the string should be n i.e _ _ _ _ and so on **n** (n spaces) so for each blank space I have 2 combinations(Because Σ is with 2 symbols) So 2x2x2x2 and so on n times = $\mathbf{2^n}$

Conclusion:

- If number of symbols of Σ={a,b}is **2**, then number of strings of length **n** that can be generated over the alphabet Σ is $\mathbf{2^n}$
- If number of symbols of Σ={a,b,c}is **3**, then number of strings of length **n** that can be generated over the alphabet Σ is $\mathbf{3^n}$
- So finally, if number of symbols of Σ is represented by **|Σ|**, then number of strings of length **n** that can be generated over the alphabet Σ is $\mathbf{|\Sigma|^n}$

1.3.2 Powers of Σ (Powers of an alphabet):

Let Σ be an alphabet, then Σ^K is nothing but set of all strings of length K, over Σ.

If Σ={a,b}

- $\mathbf{\Sigma^1}$ is nothing but set of all strings over Σ of length '1'
  $\mathbf{\Sigma^1}$ ={a,b} and total strings is: $\mathbf{|\Sigma^1|}$ = 2

- $\mathbf{\Sigma^2}$ is nothing but set of all strings over Σ of length '2'
  $\mathbf{\Sigma^2}$ => **Σ.Σ (concatenation)** => {aa, ab, ba, bb} and total strings is: $\mathbf{|\Sigma^2|}$ = 4

- $\mathbf{\Sigma^3}$ is nothing but set of all strings over Σ of length '3'
  $\mathbf{\Sigma^3}$ => **Σ.Σ.Σ (concatenation)** => {aaa, aab, aba, abb, baa, bab, bba, bbb} and total strings is: $\mathbf{|\Sigma^3|}$ = 8

1.3.3 Empty String (ε epsilon):

The empty string consisting of zero symbols, it is denoted by **ε(epsilon)**.

- $\mathbf{\Sigma^0}$ is nothing but set of all strings over Σ of length '0'
  $\mathbf{\Sigma^0}$ ={**ε**} , **ε** is called epsilon, **|ε|=0**, and total strings is: $\mathbf{|\Sigma^0| = 1}$

1.3.4 Kleene Closure (Σ*):

Set of all possible strings including epsilon(ε) over an alphabet Σ={a,b}.

- $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \ldots\ldots \Sigma^n$ i.e Union of all lengths of strings.
  Σ* = {{ε} U {a,b} U {aa,ab,ba,bb} ……} i.e All Possible Strings over Σ

1.3.5 Positive Kleene Closure (Σ^+):

Set of all possible strings excluding epsilon(ε) over an alphabet Σ={a,b}.

- $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \ldots\ldots \Sigma^n$

  Σ^+ = {{a,b} U {aa,ab,ba,bb} ……}

1.3.6 Prefix of a string:

A prefix of a string is any number of leading symbols of that string.

Ex: Let a string w=abc , then

Prefixes are: ε , a , ab and abc.

1.3.7 Suffix of a string:

A suffix of a string is any number of trailing symbols of that string.

Ex: Let a string w=abc , then

Suffixes are: ε , c , bc and abc

1.3.6 Substring:

A part of a string is called a substring.

Ex: Let a string w=abcd , then

Substrings are: a , b , c , d , ab , bc , cd , abc , bcd , and abcd.

But acd is not a substring. Similaryly ac , ad , bd , abd are not substrings.

1.4 Language:

A language is a *set of strings*, chosen from some alphabet Σ. It is denoted by L.

Or

we can say- A Language is a subset of Σ*.

- **Σ*** is anyways an infinite language but also a complete language.
- **Σ*** is the super set of all the languages i.e If L is a language then **L ⊆ Σ***

Examples:

1. Set of all strings of length 2 that can be generated over the alphabet Σ ={a,b}
   L_1 = {aa, ab, ba, bb} // This is a “Finite Language”

2. Set of all strings which start with **a** that can be generated over the alphabet Σ ={a,b}
   L_2 = {a, aa, ab, aab, aba, aaa, ……} //This is an “Infinite Language”

3. Set of all strings which end with 0, that can be generated over the alphabet Σ ={0,1}
   L_3 = {0, 00, 10, 000, 010, 100, 110, ……} //This is an “Infinite Language”

Note:

1. language which can be formed over Σ can be **Finite** Or **Infinite**.
2. L={ } that is empty set is also a valid language, it is called **empty language.** It is denoted by Φ(phi).
3. L={ε} that is the set consisting of empty string(epsilon) is also a valid language, but it is not empty language.

1.5 Finite Automata/Finite State Machine:

Automata:

An automaton (Plural: automata) is an “abstract computing machine/device” (simply called a mathematical model), i.e something that works automatically or a self-operating machine/device.

Ex: ATM, Cola-Machine, etc.

Finite Automata (FA):

Finite automata or finite state machine is an abstract machine (a mathematical model) which contains finite number of states and transitions. Basically, it is the simplest machine of digital computer to recognize patterns(languages). It is a recognizer for regular languages.

Or

Finite automata or finite state machine is an abstract machine (a mathematical model) which have five tuples or elements. It has a set of states and rules for moving from one state to another state/s (is called transition) but it depends upon the applied input symbol.

Note:

- Abstract means existing in thought or as an idea but not having a physical existence.
- A device need not even be a physical hardware.
- An automaton with a finite number of states is called a Finite Automaton (FA) or Finite State Machine (FSM).

Formal Definition of Finite Automata:

An automaton can be represented by a 5-tuples (Q, ∑, δ, q0 , F) and denoted by M.

Where:

1. Q is a finite set of states.
2. ∑ is a finite set of input symbols, called the alphabet of the automaton.
   ∑ is called Sigma.
3. δ is the transition function. δ is called Delta.
   Transition Function is defined as:
   - Transition Function takes a state and input symbol as arguments, then returns a state/states.
   - Ex: δ(q,a) = p, where q and p are states, a is an input symbol.
   - i.e if the FA is in state q, and input a is received, then the FA goes to state p.
4. q0 is the initial state/starting state from where any input is processed (q0 ∈ Q).
5. F is a set of final state/states (F ⊆ Q).

Classification of Finite Automata:

Finite automata can be classified into two types:

1. Finite Automata without output: we have three categories.
   - Deterministic Finite Automata (DFA).
   - Non-Deterministic Finite Automata (NFA or NDFA).
   - Non-Deterministic Finite Automata with epsilon moves (ε-NFA or ε-NDFA).
2. Finite Automata with output: we have two categories.
   - Moore machine.
   - Mealy machine.

Finite Automata Notations:

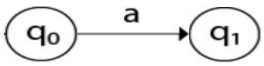
The FA can be represented in two ways:

1. Transition Diagram
2. Transition Table

Transition Diagram: A transition diagram or state diagram is a directed graph which can be constructed as follows:

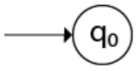
- The vertices represent the states, which is denoted by the circle.

- The arcs(arrows) labeled with an input symbol show the transitions.



Transition from one state to another

- The initial state/starting state is denoted by circle with an empty single incoming arc.

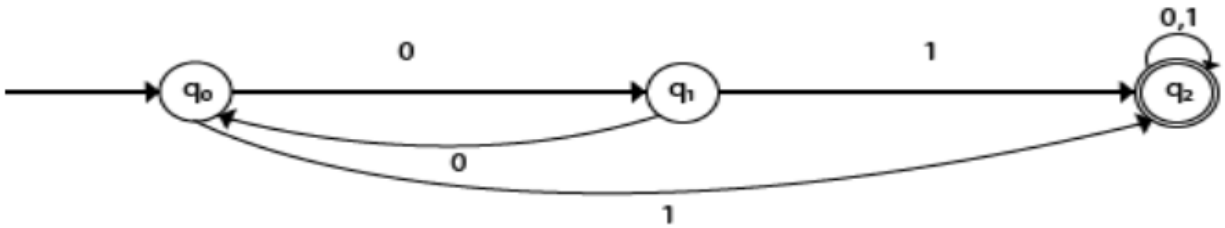


Start state

- The final state is indicated by double circles.

Final state

Example: Transition Diagram



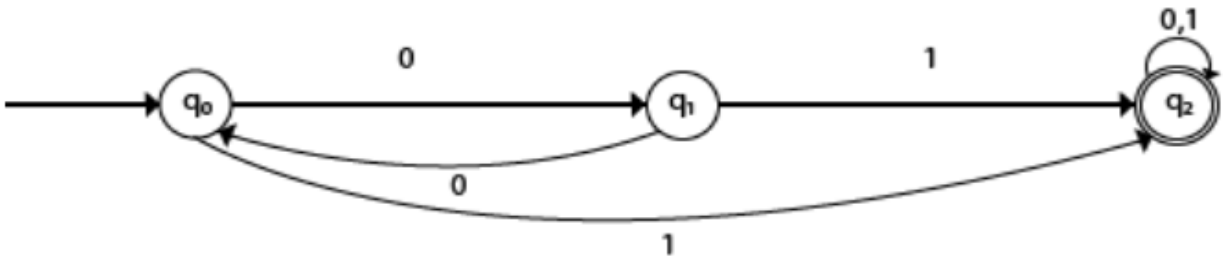
Transition Table: A transition table is basically a tabular representation of the transition functions. It takes two arguments (a state and a symbol) and return a state (next state).

A transition table is represented by the following things:

- The rows of the table correspond to the states.
- The columns correspond to the input symbols.
- The entry for the row corresponding to state q and the column corresponding to input a is the next state.
- The starting state is denoted by an arrow.
- The final state is denoted by a star or a circle.

Example:

Transition Diagram:



Transition Table: Transition table for the above transition diagram is as follows:

Present State	Next state for Input 0	Next State of Input 1
→q0	q1	q2
q1	q0	q2
*q2	q2	q2

In the above q0 is initial state, q2 is final state and q1 is intermediate state.

Acceptance of String:

Given a string 'w', we can determine whether the string 'w' is accepted by the Finite Automata (DFA/NFA) or not.

Starting from the initial state by scanning the characters in w once, one by one from left to right, after scanning the whole string, if we reach to final state then we can say the string 'w' is "accepted" by the FA(DFA/NFA), otherwise we can say the string 'w' is "rejected".

Acceptance of Language (Set of Strings):

Given a language 'L', we can determine whether the language 'L' is accepted by the Finite Automata (DFA/NFA) or not.

Language means set of strings. So, if every string in the language 'L' is accepted by the FA(DFA/NFA), then we can say the language 'L' is "accepted" by the FA(DFA/NFA).

If any string 'w', which in language 'L' is rejected by the FA(DFA/NFA) then we can say the language 'L' is not accepted by FA(DFA/NFA).

The language 'L' accepted by the finite automata 'M' is denoted by L(M).

1.6 Deterministic Finite Automata (DFA):

It is a Finite Automata in which from every state on every input symbol exactly one transition should exist.

In DFA, for each input symbol, one can determine the state to which the machine will move. Hence, it is called Deterministic Automaton. As it has a finite number of states, the machine is called Deterministic Finite Automaton.

DFA consists of 5-tuples of machine M = (Q, ∑, δ, q0 , F)

where:

1. Q is a finite set of states.
2. ∑ is a finite set of input symbols, called the alphabet of the automaton.
   ∑ is called Sigma.
3. δ is the transition function. δ is called Delta.
   Transition Function is defined as δ : Q x ∑ → Q
   - Transition Function takes a state and input symbol as arguments, then returns a state.
   - Ex: δ(q,a) = p, where q and p are states, a is an input symbol.
   - i.e if the FA is in state q, and input a is received, then the FA goes to state p.
4. q0 is the initial state/starting state from where any input is processed (q0 ∈ Q).
5. F is a set of final state/states (F ⊆ Q).

Representation of a DFA: A DFA is represented by Transition Diagram or Transition Table.

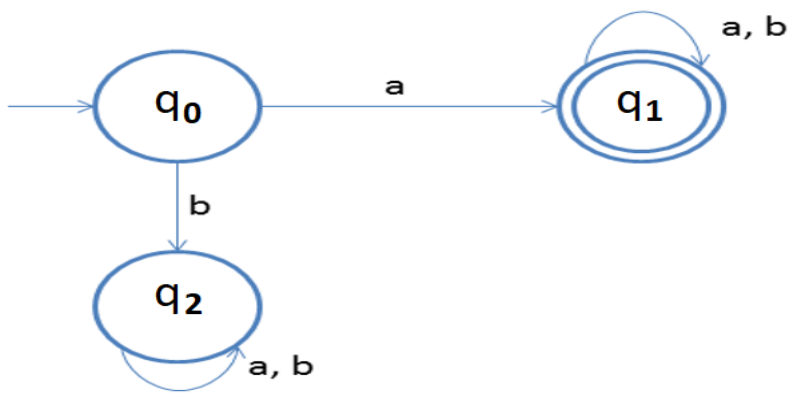
Example1: Design a DFA which accept set of all strings over an input alphabet Σ={a,b}, that start with 'a'.

Ans: The given language is set of all strings which start with 'a'.

i.e L = {a, aa, ab, aaa, aab, aba, abb, aabb, aaabbba, …..}

Clearly the language is infinite because there is infinite number of strings.

Transition Diagram:



Where q0 is initial state, q1 is final state, and q2 is dead state or trap state.

The idea is very simple, follow the steps below and you will understand.

1. We know that first input must be ‘a’ and from start state on ‘a’ we should accept so go to final state, so two states are confirmed: start state and final state i.e q0 and q1.
2. Once we reach final state, we should accept everything that is why we have a loop on ‘a’ and ‘b’ inputs.
3. And if the first input is something else than ‘a’ then string should not be accepted. So from start state on ‘b’ we should go to dead state or trap state, third state is confirmed i.e q2.
4. Once we reach q2, we don’t care whatever comes on state q2 as state q2 now will work as dead state.

Testing: Lets take one input string w=abab, and check whether it is accepted or not by the designed DFA.

1. Start from the initial state i.e q0
2. First input is ‘a’, so from state q0 on input ‘a’, we will go to state q1
3. Second input is b, so from present state q1 on input ‘b’, we will go to state q1 itself
4. Third input is a, so from state q1 on input ‘a’, we will go to state q1 itself
5. Fourth input is b(string is completed), so from state q1 on input ‘b’, we will go to state q1 itself**(final state)**

After end of the string w=abab, we are at final state, so string is accepted.

1.7 Non-Deterministic Finite Automata (NFA or NDFA):

The term non-determinism means from a given state and on an input symbol, the transition function δ may lead to a set of states which is subset of Q.

i.e., A state have more than one transition on the same input symbol**.**

Formally NFA can be represented by 5 tuples M=(Q , ∑ , δ , q0 ,F)

Where:

1. Q- non empty finite set of states
2. ∑- finite no of input symbols
3. δ- transition function that is defined as $\delta : Q \times \sum \rightarrow 2^Q$
4. q0-initial state
5. F- set of final states

Acceptance of String by NFA:

- In DFA, string is said to be accepted if finite control stops at a final state after processing entire string.
- Since there is only one path, one can confirm for acceptance.
- In case of NFA, for a string 'w' there may be multiple paths exist for the given string and some paths may lead to final state and some may not.
- If we find a path(at least one) which leads to a final state after processing entire string then we can say the string 'w' is accepted by NFA.

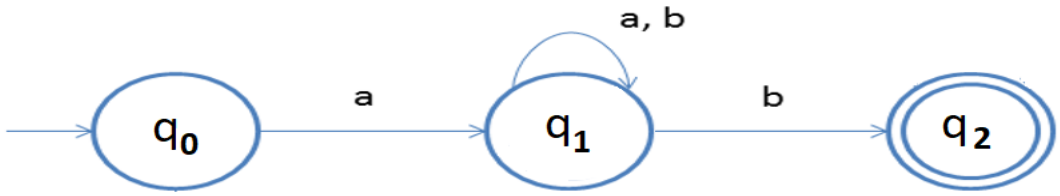
Example1: Design a NFA which accept set of all strings over an input alphabet Σ={a,b}, that start with 'a' and end with 'b'.

Ans: The given language is set of all strings over an alphabet Σ={a,b}, that start with 'a' and end with 'b'.

i.e L = {ab, aab, abb, aaab, aabb, abab, abbb, aaabb, aababbab, ……..}

Clearly the language is infinite because there is infinite number of strings.

Transition Diagram:



Where q0 is initial state, q1 is intermediate state, and q2 is final state.

The idea is very simple, follow the steps below and you will understand.

1. We know that first input must be 'a' and from start state q0 on 'a' we should not accept because we must see last input 'b', so go to q1(intermediate state).
2. We know that last input must be 'b', so from q1 on 'b' we must accept so go to final state i.e q2.
3. We should accept everything in between first input 'a' and last input 'b', that is why we have a loop on 'a' and 'b' inputs at q1 state.
4. And if the first input is something else than 'a' we don't care, so no transition on input 'b' from q0 state. And also, we don't care about transitions on inputs 'a' and 'b' from q2 state.

Limitations of NFA:

- It cannot be implemented because of non-deterministic in nature. i.e from the state on an input symbol, there may be multiple transitions.
- To overcome this, it must be converted back to Deterministic FA which is implementable.

Note:

1: Every DFA is NFA but vice-versa is not true.
The above property is true because the δ of DFA is ⊆ δ of NFA
As δ : Q X Σ -> 2Q ⊇ Q X Σ -> Q

2: We do not have dead states in NFA.

3: NFA and DFA both are equivalent in power.
This is because "Every DFA is NFA" and "We can convert NFA to DFA"

1.8 Conversion from NFA to DFA:

Equivalence of Deterministic and Non Deterministic Finite Automata:

- Every language that is described by some NFA can also be described by some DFA.
- A DFA can have 2^n states, while the NFA for the same language has n states.
- The subset construction method is used to prove the equivalence of DFA and NFA.
- The subset construction starts from an NFA N = $(Q_N, \sum, \delta_N, q_0, F_N)$ and constructs its equivalent DFA D = $(Q_D, \sum, \delta_D, \{q_0\}, F_D)$.
  - Q_D is subsets of Q_N where Q_D has 2^n states in the worst case.
  - $\{q_0\}$ is the initial state of Q_D .
  - F_D is the set of all final states that contain the final states of Q_N.
  - δ_D is denoted as $Q_D \times \sum \rightarrow Q_D$

  where $\delta_D\ (\{q_0\}, x) = [q_1, q_2, q_3, \ldots.. q_j]$ iff

  $$\delta_N\ (q_0, x) = \{q_1, q_2, q_3, \ldots.. q_j\}$$

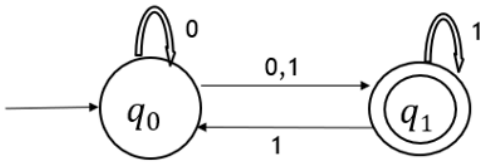
Steps for the Conversion of NFA to DFA:

1. The start state of NFA N will be the start state of DFA D. So, we add q_0 of NFA to DFA and finds the transitions from the q_0 state.
2. For each state in Q_D the transitions for each input symbol ∑ has to be obtained.
3. If the transition goes to more than one states make them as single new state.
4. If the output of the transition is a new state that is not in the Q_D of DFA then add it to Q_D.
5. Find the transitions for the new state to the input symbols ∑.
6. Steps 3 and 4 are repeated until no new state is generated.

Example: Conversion of NFA to DFA

- Let M=({q_0, q_1},{0,1},δ, q_0,{q_1}) be NFA where δ($q_0, 0) = \{q_0, q_1\}$, δ($q_0, 1) = \{q_1\}$, δ($q_1, 0) = \emptyset$,δ($q_1, 1) = \{q_0, q_1\}$. Construct the equivalent DFA.

Solution: Let us draw a transition diagram & table for the given NFA



Transition Diagram

States	Input 0	Input 1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_1\}$
$* q_1$	$\emptyset$	$\{q_0, q_1\}$

Transition Table

Let the DFA M'=(Q',∑,δ', q_0',F')

Step 1:The start state of NFA q_0 is the start state of DFA

Step 2:Computing the transition function δ' for the DFA

As δ($q_0, 0) = \{q_0, q_1\}$, δ'([q_0],0) $= [q_0, q_1]$

δ'([q_0],1) $= [q_1]$

Step 3: Add the new states to the Q'of DFA i.e Q'={[q_0],[q_1],[q_0, q_1]}

Step 4: Find the transitions for the new state with the input symbols ∑.

δ'([q_1],0) $= \emptyset$

δ'([q_1],1) $= [q_0, q_1]$

δ'([q_0, q_1], 0) $= [q_0, q_1]$

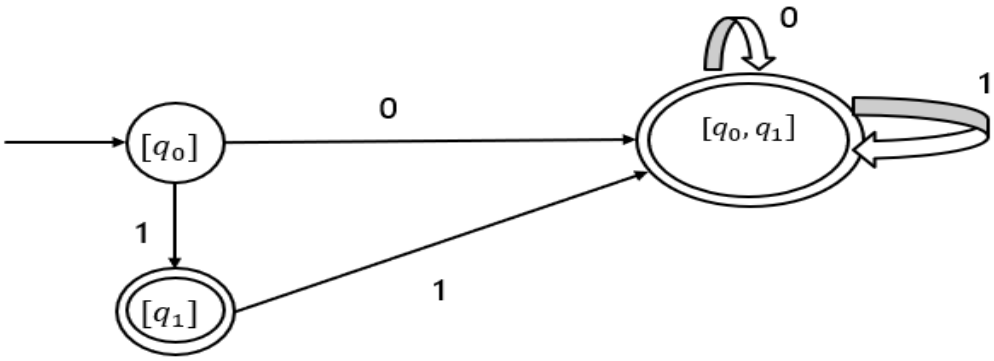
δ'([q_0, q_1], 1) $= [q_0, q_1]$.

Step 5: No new state is obtained from the above transition. So we will stop the process and an equivalent DFA is obtained.

Transition Table for the DFA

states	Input 0	Input 1
$->q_0$	$[q_0, q_1]$	$[q_1]$
$[q_1]$	$\emptyset$	$[q_0, q_1]$
$[q_0, q_1]$	$[q_0, q_1]$	$[q_0, q_1]$

Transition Diagram of Equivalent DFA



1.9 ε-NFA (NFA with ε-moves):

- Transition from one state to another without reading any input symbol is known as ε-moves. i.e transition function is δ(q, ε)=q
- However, to distribute the complexity of the design or enhance understanding of the reader, the NFA can be extended by including transitions on 'ε'
- The final automaton doesn't include 'ε' transitions
- Formally NFA with ε-moves is a 5-tuple machine M=(Q,∑, $\boldsymbol{\delta}$**,** q0, F) where
    1. Q- finite no of states
    2. ∑-finite no of input symbols
    3. $\boldsymbol{\delta}$-transition function Q × ∑ U { ε } → 2^Q

       i.e., δ(q , a) consisting of a state p such that there is a transition labeled 'a' from q to p, where a is either ε or input symbol
    4. q0-initial state
    5. F-final state

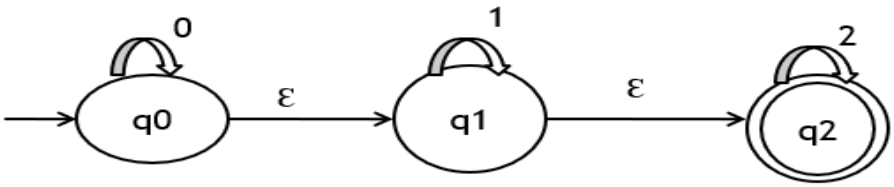
Example1: Design a ε-NFA (NFA with ε-moves) which accept set of all strings over an input alphabet Σ={0,1, 2}, that any number of 0's followed by any number of 1's followed by any number of 2's.

Ans: The given language is set of all strings over an alphabet ={0,1, 2}, that any number of 0's followed by any number of 1's followed by any number of 2's.

i.e L = { ε , 0, 1, 2, 00, 01, 02, 11, 12, 22, 011, 012, 0012, ……..000111122,…...}

Clearly the language is infinite because there is infinite number of strings.

Transition Diagram:



Transition Table:

	0	1	2	ε
->q0	q0	Φ	Φ	q1
q1	Φ	q1	Φ	q2
*q2	Φ	Φ	q2	Φ